Quasi-Optimal Learning based Analytical Powered Descent Guidance with State and Action Constraints

Suraj Kumar * [#], Vaibhav Agrawal[†#],Aditya Rallapalli[‡],Nivrti Priyadarshini[§], Bharat Kumar GVP[¶]
U R Rao Satellite Centre, Indian Space Research Organization (ISRO), Bangalore 560017 India

This paper proposes a Quasi-Optimal Learning based Powered Descent Guidance (QO-LPDG) framework that combines the analytical structure of the optimal feedback guidance with data-driven learning to ensure quasi-optimal fuel performance under state and action constraints. The guidance acceleration is parameterized using a function form inspired by optimal feedback guidance law. Unlike existing methods that estimate the remaining time of flight only at the start of the guidance phase, a surrogate learning-based model is developed to estimate it continuously along the trajectory. This model is trained using simulation-based optimization to ensure that the generated trajectories satisfy system constraints and selects a fuel-optimal solution for each initial condition from a set of trajectories defined by the chosen function class. The method is termed quasi-optimal because it finds the best solution only within the space of trajectories parameterized by the chosen acceleration function. The advantage of computing remaining time-of-flight continuously, rather than once at the start of the guidance phase, is especially demonstrated under perturbed conditions. Additionally, the proposed method increases the controllable region compared to the classical guidance algorithm.

Nomenclature

r	=	position vector of spacecraft with respect to target landing site
v	=	velocity vector of spacecraft
g	=	gravitational acceleration vector of the planet
a	=	net acceleration vector due to propulsion system
m	=	mass of the spacecraft
T_c	=	magnitude of total thrust from propulsion system
α	=	positive constant associated with fuel consumption rate
t_0	=	initial time instant
t_f	=	time of flight
r_0	=	initial value of position vector
v_0	=	initial value of velocity vector
r_f	=	terminal value of position vector
v_f	=	terminal value of velocity vector
a_f	=	terminal value of acceleration vector
m_{wet}	=	initial mass of the spacecraft
ρ_m	=	magnitude of minimum thrust from the propulsion system
ρ_M	=	magnitude of maximum thrust from the propulsion system
Δ_T	=	upper bound on rate of change of thrust
θ	=	glideslope angle
θ_{lim}	=	upper bound on glideslope angle
λ_r	=	costate vector of position vector
λ_v	=	costate vector of velocity vector

*Scientist/Engineer, Controls and Digital Area; e-mail: surajk@ursc.gov.in(Corresponding Author), suraj27avionics@gmail.com
†Scientist/Engineer, Controls and Digital Area; e-mail: avaibhav@ursc.gov.in
‡Scientist/Engineer, Controls and Digital Area; e-mail: adityar@ursc.gov.in
§Scientist/Engineer, Controls and Digital Area; e-mail: nivriti@ursc.gov.in
¶Scientist/Engineer, Controls and Digital Area; e-mail: bharat@ursc.gov.in
#These authors contributed equally to this work

t_{go}	=	time-to-go
t	=	current time
C_0, C_1, C_2	=	coefficients of polynomial expression used to parameterize acceleration vector
π	=	mapping between spacecraft state vector and t_{go}
$\mathcal{S}(t)$	=	spacecraft state vector at time t
$\mathcal{F}(S)$	=	set of controllable t_{go} and corresponding fuel consumption for given state vector $\mathcal{S}$
$t^*_{go}(S)$	=	fuel-optimal t_{go} from the set $\mathcal{F}(S)$ for given state vector $\mathcal{S}$
Σ	=	covariance matrix representing 3σ dispersions in position and velocity
$\mathcal{X}_{init}$	=	set of 3σ perturbed state vectors
$\mathcal{D}_0$	=	initial perturbation dataset
$\mathcal{D}_r$	=	trajectory rollout dataset
$\mathcal{D}$	=	complete dataset for training
$\hat{t}_{go}$	=	time-to-go estimate produced by neural network
$h^{(i)}$	=	output of nodes in the ith layer
$W^{(i)}$	=	weight vector of ith layer
$b^{(i)}$	=	bias term of ith layer
$\phi^{(i)}$	=	non-linear activation function of ith layer
λ	=	weight decay coefficient for L2 regularization term
Δ_y	=	perturbation in downrange
Δ_z	=	perturbation in altitude
Δ_{vy}	=	perturbation in horizontal velocity
Δ_{vz}	=	perturbation in vertical velocity

I. Introduction

Autonomous guidance and control technology plays an important role in soft landing in planetary missions. A typical planetary landing profile comprises two phases: the main braking phase and the terminal descent phase. The main braking phase is responsible for reducing the bulk of the orbital velocity and traveling the maximum distance to the designated landing site, while the terminal phase is responsible for pinpoint landing. For lunar landing, both the main and terminal phases are executed using the propulsion system. In contrast, planetary bodies like Mars possess a thin atmosphere, allowing the initial phase to use aerodynamic drag via parachutes, followed by a propulsive terminal phase. Regardless of atmospheric conditions, the terminal descent phase remains crucial as it must ensure a precise and safe descent while handling constraints related to state and control. With increasing interest in autonomous landings on the Moon, Mars, and other planetary bodies[1–3], robust guidance strategies have become more critical than ever.

Closed-loop Powered Descent Guidance (PDG) computes the required braking acceleration—both thrust magnitude and direction—as well as the time of flight to reach the desired terminal state, in real time, using navigation feedback. In addition to the physical characteristics of the resulting trajectory, the reliability and computational complexity of the onboard algorithm are critical for practical guidance applications. However, there are inherent trade-offs between fast execution time, robustness, propellant optimality, and the ability to handle constraints effectively. Broadly, powered descent guidance techniques can be categorized into two main classes: analytical guidance and computational guidance. Several analytical approaches have been proposed in the literature, with early developments tracing back to the Apollo guidance framework. Apollo-type polynomial guidance algorithms, which use linear or quadratic time-based polynomials, offer analytical closed-loop solutions[4],[5],[6],[7]. Augmented Apollo Powered Descent Guidance(APDG) is presented in [8] where the user can choose from a family of tunable guidance laws of APDG variants by selecting gain. While these methods are attractive for their simplicity and accuracy, they generally lack the ability to enforce thrust or flight constraints. Zero-Effort-Miss/Zero-Effort-Velocity (ZEM/ZEV) feedback guidance computes optimal acceleration minimizing control effort to achieve desired terminal position and velocity[9],[10],[11]. However, it generally does not account for control and state constraints, such as thrust magnitude limits, glide-slope constraints, or thrust-pointing restrictions. Computational guidance approaches solve an optimization problem online to generate control commands. These methods are broadly classified into two categories: indirect methods, which rely on solving the necessary conditions of optimality, and direct methods, which transcribe the optimal control problem into a nonlinear programming problem for numerical solution. Guidance techniques based on indirect methods [12],[13] utilizes the application of the Pontryagin Maximum Principle (PMP) to solve the two-point boundary value problem that arises from the necessary conditions of optimality. While these methods can handle control constraints like thrust

magnitude limits and thrust-pointing constraints, they often require a good initial guess to ensure smooth convergence. Additionally, enforcing state constraints can still be difficult. Guidance algorithms based on direct methods have gained popularity in recent years, driven by advances in efficient optimization solvers and increased onboard computational capabilities. Convex optimization [14], [15],[16] and pseudo-spectral methods [17, 18] compute fuel-optimal control problem with state and control constraint by directly transcribing the original problem into discrete state and action space and solving them with specific solvers. Although this formulation explicitly handles a range of control and state constraints—such as thrust pointing, thrust magnitude limits, and glide-slope requirements—its main limitation is the high computational load, which poses challenges for execution on low-speed onboard processors that are much slower than commercial CPUs. Additionally, the guidance module must share computational resources with other critical subsystems such as navigation, control, sensor processing, and telemetry, further limiting the available onboard processing time. Learning-based guidance approaches [6, 19–24] offer the best of both worlds by explicitly learning complex components offline using simulation-generated data—components that are otherwise computationally intensive to solve in real time. This enables the system to produce solutions that are near-optimal under state and control constraints while retaining an analytic structure suitable for efficient onboard implementation.

This paper proposes a Quasi-Optimal Learning based Powered Descent Guidance (QO-LPDG) framework that combines the analytical structure of optimal feedback guidance law with data-driven learning to ensure quasi-optimal fuel performance under state and action constraints. The proposed framework introduces a learned surrogate model that continuously estimates the remaining time-to-go along the trajectory under realistic state and action constraints. This adaptive capability allows the guidance system to respond to mid-trajectory perturbations or uncertainties, such as partial actuation failures or navigation errors, thereby enhancing robustness and reliability during powered descent. Additionally, the proposed method increases the controllable region compared to the existing analytical guidance algorithm.

The paper is organized as follows. Section II sets up the powered descent landing problem for the terminal phase of the planetary landing. Section III reviews the classical optimal guidance problem. Section IV details the formulation of the proposed QO-LPDG guidance algorithm. Section V presents extensive simulation results. Section VI concludes the paper.

II. Problem Formulation

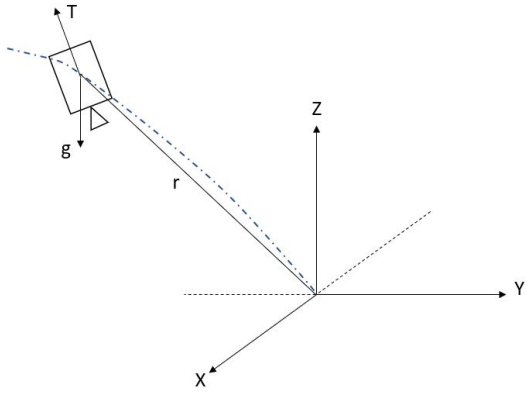


Fig. 1 Coordinate System

The proposed algorithm is developed for the terminal phase of the powered descent sequence of planetary landing and demonstrated for the soft landing scenario of Mars. It is assumed that either there is no atmosphere, as in the case of lunar terminal descent, or that aerodynamic forces can be safely ignored, as in Mars terminal descent. The problem is described in local surface-fixed frame centered at the target landing site as shown in Fig 1. Axis x, y, z represents the direction of lateral, horizontal and vertical motion respectively. The spacecraft decoupled translational dynamics is

considered in the local surface-fixed frame under the non-rotating planet assumption as

$$\begin{aligned} \dot{\boldsymbol{r}} &= \boldsymbol{v} \\ \dot{\boldsymbol{v}} &= \boldsymbol{g} + \boldsymbol{a} \\ ||\boldsymbol{a}|| &= \frac{T_c}{m} \\ \dot{m} &= -\alpha T_c \end{aligned} \tag{1}$$

where $\boldsymbol{r} \in \mathbb{R}^3$ and $\boldsymbol{v} \in \mathbb{R}^3$ are the position and velocity vectors relative to the target landing site; $\boldsymbol{a} \in \mathbb{R}^3$ is the net acceleration vector due to the propulsion system; m is the mass of the spacecraft; $\boldsymbol{g} \in \mathbb{R}^3$ is the constant gravitational acceleration vector of the planet; α is the positive constant associated with the fuel consumption rate; T_c is the magnitude of the total thrust from the propulsion system.
The boundary conditions associated with the problem are given as

$$\begin{aligned} \boldsymbol{r}(t_0) = \boldsymbol{r_0}, \quad & \boldsymbol{r}(t_f) = \boldsymbol{r_f} \\ \boldsymbol{v}(t_0) = \boldsymbol{v_0}, \quad & \boldsymbol{v}(t_f) = \boldsymbol{v_f} \\ \boldsymbol{a}(t_0) = \text{free}, \quad & \boldsymbol{a}(t_f) = \boldsymbol{a_f} \\ m(t_0) = m_{wet}, \quad & m(t_f) = \text{free} \end{aligned} \tag{2}$$

where $\boldsymbol{r_0}, \boldsymbol{v_0}$ are the initial values of the position and velocity vectors; $\boldsymbol{r_f}, \boldsymbol{v_f}$ and $\boldsymbol{a_f}$ are the terminal values of the position, velocity, and acceleration vectors; m_{wet} is the initial mass of the spacecraft. The terminal acceleration constraint ensures vertical orientation of the spacecraft at the final time.
The total thrust magnitude is limited within the allowable operating range of the propulsion systems as

$$0 \leq \rho_m \leq T_c \leq \rho_M \tag{3}$$

$$\dot{T_c} \leq \Delta_T \tag{4}$$

where ρ_m, ρ_M denote the minimum and maximum thrust; Δ_T denotes the upper bound on the rate of change in thrust. Eq.(3) and (4) are associated with the physical limits on the thrust and thrust rate constraints by the propulsion system respectively. The state constraints associated with the problem are given as

$$r_3(t) \geq r_f(3) \quad \forall t \in [0, t_f] \tag{5}$$

$$\theta(t) \leq \theta_{lim} \tag{6}$$

where r_3 is the vertical component of the current position vector; $r_f(3)$ is the vertical component of the terminal position vector; $\theta(t)$ is the glideslope angle. Eq.(5) and (6) are associated with the sub-target travel and pitch angle constraint respectively.

III. Classical Optimal Guidance Formulation and its Limitations

The global solution of the minimum fuel guidance problem can be efficiently computed by solving an optimal control problem subject to dynamics, state, and control constraints. Even though such a formulation is quite useful for trajectory optimization and mission planning, the numerical and iterative nature of such a solution limits its applicability onboard for low-speed computers. In such scenarios, analytical explicit guidance is preferred. Analytical optimal feedback guidance algorithm can be arrived using calculus of variation under reasonable set of approximations in state/control constraints and dynamics. Here, we summarize the classical optimal guidance[9] and present key insights from the algorithm that forms the basis of our design.
Consider the classical optimal control problem with cost function as

$$J = \Gamma t_f + \frac{1}{2} \int_{t_0}^{t_f} \boldsymbol{a}^T \boldsymbol{a} \, dt \tag{7}$$

subject to dynamics constraints 1, state and control constraints 3-6 with boundary conditions on position and velocity given by

$$\begin{aligned} \boldsymbol{r}(t_0) &= \boldsymbol{r_0}, \boldsymbol{r}(t_f) = \boldsymbol{r_f} \\ \boldsymbol{v}(t_0) &= \boldsymbol{v_0}, \boldsymbol{v}(t_f) = \boldsymbol{v_f} \end{aligned} \tag{8}$$

Suppose that the mass flow rate in equation 1, and constraints 3-6 are ignored. $\boldsymbol{g}$ is also assumed constant, as is the case for a typical terminal descent phase. Then, the optimal control equation can be derived using the necessary condition of optimality via the calculus of variations. The cost function minimizes the weighted combination of time of flight and control effort. The Hamiltonian is given as

$$H = \frac{1}{2}\boldsymbol{a}^T\boldsymbol{a} + \boldsymbol{\lambda_r}^T\boldsymbol{v} + \boldsymbol{\lambda_v}^T(\boldsymbol{g} + \boldsymbol{a}) \tag{9}$$

where $\boldsymbol{\lambda_r}$ and $\boldsymbol{\lambda_v}$ are the costate vectors associated with position and velocity vectors respectively. Let $t_{go} = t_f - t$. Using necessary conditions of optimality, guidance acceleration is given as

$$\boldsymbol{a} = \frac{6[\boldsymbol{r_f} - (\boldsymbol{r} + t_{go}\boldsymbol{v})]}{t_{go}^2} - \frac{2(\boldsymbol{v_f} - \boldsymbol{v})}{t_{go}} - \boldsymbol{g} \tag{10}$$

Using the transversality condition, the quartic expression involving t_{go} is as follows,

$$(\Gamma + \frac{1}{2}\boldsymbol{g}^T\boldsymbol{g})t_{go}^4 - 2t_{go}^2(\boldsymbol{v_f}^T\boldsymbol{v_f} + \boldsymbol{v}^T\boldsymbol{v} + \boldsymbol{v_f}^T\boldsymbol{v}) + 12t_{go}(\boldsymbol{r_f} - \boldsymbol{r})^T(\boldsymbol{v} + \boldsymbol{v_f}) - 18(\boldsymbol{r_f} - \boldsymbol{r})^T(\boldsymbol{r_f} - \boldsymbol{r}) = 0 \tag{11}$$

Eq. 11 can be solved analytically to obtain the value of time-to-go. When $\Gamma = 0$, the guidance law is referred to as E-guidance which is proven to be not fuel optimal in nature as it optimizes $\frac{1}{2}\int_{t_0}^{t_f} \boldsymbol{a}^T\boldsymbol{a}\, dt$ (minimum fuel objective function is given by $\frac{1}{2}\int_{t_0}^{t_f} ||\boldsymbol{a}||\, dt$).

To summarize, optimal guidance formulation consists of two key components: the computation of guidance acceleration and the time-to-go(t_{go}). We observe that the resultant form of guidance acceleration takes the form of a polynomial in t_{go} where both the coefficients of the polynomial and t_{go} are computed using the current state feedback from the navigation system. In fact, the order of the polynomial is determined by the initial and terminal boundary constraints. For instance, the initial and terminal constraints in the position and velocity in the optimal guidance resulted in a first-order polynomial.

Several approaches for computing t_{go} have been reported in the literature, starting with the Apollo guidance framework. However, existing methods have three major limitations. First, they rely on simplifying assumptions about the system dynamics or trajectory. For instance, in the Apollo guidance law[4], t_{go} is chosen such that the vertical component of the acceleration varies linearly with time. In the optimal feedback guidance framework described in the previous section, t_{go} is derived from the transversality conditions using the calculus of variations, based on a simplified optimal control problem that ignores thrust constraints and mass flow dynamics.

The second limitation lies in recomputation of t_{go} under perturbed conditions throughout the trajectory. While the quartic formulation may be valid throughout the trajectory under its original assumptions, many methods compute t_{go} only once at the start and decrement it during each guidance cycle. The third limitation is the lack of explicit fuel optimization in most existing methods. To the best of the author's knowledge, incorporation of a fuel-consumption objective was first reported in [6] within the polynomial guidance framework using deep reinforcement learning. However, this approach still computes t_{go} only at the start of the guidance and updates it by simple decrementing the same in each cycle, without recomputation based on current state feedback.

Our approach addresses these limitations by learning a surrogate model for t_{go} using supervised learning that is used continuously throughout the trajectory. This model is trained via simulation data to ensure that fuel optimality and constraint satisfaction are explicitly considered in the design.

IV. QO-LPDG Formulation

We can now interpret the solution to the optimal feedback guidance problem with the following argument which forms the basis of QO-LPDG Formulation. First, we consider the feasibility problem of finding a feedback guidance solution that satisfies both the system dynamics and boundary constraints. This can be achieved by using a polynomial of sufficient order to meet the boundary conditions. The polynomial coefficients are then computed to satisfy the dynamics, resulting in a control law that behaves like feedback guidance. However, this solution is generally not optimal in terms of fuel performance. To improve it, we introduce free variables into the guidance that can be optimized. In

the case of polynomial guidance, free variables for optimization can come from choosing the polynomial order and the time-to-go t_{go}. By increasing the order of the polynomial, we can introduce more free variables. In this work, we introduce free variables only through t_{go} to perform the optimization for fuel consumption. Unlike earlier approaches, t_{go} is not treated as a single scalar value computed only at the start of the guidance phase. Instead, it is formulated as a policy that is continuously evaluated throughout the trajectory as a function of the current guidance states.

Considering the boundary constraints given by initial position, velocity and terminal position, velocity and acceleration, we prescribe a second order polynomial form for the net acceleration

$$\bar{\boldsymbol{a}}(t) = \boldsymbol{C}_0 + \boldsymbol{C}_1 t + \boldsymbol{C}_2 t^2 \tag{12}$$

Integrating the above acceleration profile leads to

$$\boldsymbol{v}(t) = \boldsymbol{v_o} + \boldsymbol{C}_0 t + \boldsymbol{C}_1 \frac{t^2}{2} + \boldsymbol{C}_2 \frac{t^3}{3} \tag{13}$$

$$\boldsymbol{r}(t) = \boldsymbol{r_o} + \boldsymbol{v_o} t + \boldsymbol{C}_0 \frac{t^2}{2} + \boldsymbol{C}_1 \frac{t^3}{6} + \boldsymbol{C}_2 \frac{t^4}{12} \tag{14}$$

The polynomial coefficients are obtained by enforcing the dynamics and boundary conditions through a series of linear systems of equations.

$$\begin{bmatrix} 1 & t_{\text{go}} & t_{\text{go}}^2 \\ t_{\text{go}} & \frac{t_{\text{go}}^2}{2} & \frac{t_{\text{go}}^3}{3} \\ \frac{t_{\text{go}}^2}{2} & \frac{t_{\text{go}}^3}{6} & \frac{t_{\text{go}}^4}{12} \end{bmatrix} \begin{bmatrix} C_0^i \\ C_1^i \\ C_2^i \end{bmatrix} = \begin{bmatrix} \boldsymbol{a}^i{}_f \\ \boldsymbol{v}^i{}_f - \boldsymbol{v_0}^i \\ \boldsymbol{r}^i{}_f - \boldsymbol{r_0}^i - \boldsymbol{v_0}^i \cdot t_{\text{go}} \end{bmatrix} \tag{15}$$

Index i represents the indices of the x, y, z components of the vector. The resultant guidance acceleration from propulsion system in computed by solving the set of linear equation and subtracting the gravity vector as

$$\boldsymbol{a}(t) = -\frac{6}{t_{go}}(\boldsymbol{v_f} - \boldsymbol{v(t)}) + \frac{12}{t_{go}^2}(\boldsymbol{r_f} - \boldsymbol{r}(t) - \boldsymbol{v}(t) t_{go}) + \boldsymbol{a_f} - \boldsymbol{g} \tag{16}$$

where $\boldsymbol{r}(t)$, $\boldsymbol{v}(t)$ is obtained from navigation system. As t_{go} approaches zero, the guidance computation approaches singularity. To prevent this, the computation of guidance coefficients is terminated once t_{go} reaches a predefined threshold. However, the execution of the guidance continues until t_{go} reaches zero using the last computed coefficients. This strategy helps minimize terminal errors.

A. Learning based t_{go} formulation

Our goal is to model the relation between t_{go} and spacecraft state vector that minimizes fuel consumption across the function space parameterized by quadratic function. Let π denote the mapping between input and output as

$$t_{go} = \pi(\boldsymbol{r}(t), \boldsymbol{v}(t)) \tag{17}$$

where π represents the policy that is learned through data via supervised learning framework. The overall dataset generation process for learning comprises of two main stages:

1) **Initial Perturbation Dataset Stage**: In this stage, the dataset is generated by applying perturbations to the initial conditions at the start of the guidance phase. For each perturbed initial state, the corresponding fuel-optimal t_{go} is computed. This stage is sufficient if the objective is to estimate t_{go} only at the start of the guidance phase and then decrement it linearly until it reaches zero.
2) **Trajectory Rollout Dataset Stage**: In this stage, the dataset is augmented by including intermediate states obtained from trajectory rollouts using the fuel-optimal t_{go} from the first stage. For each intermediate state along the trajectory, the corresponding fuel-optimal t_{go} is computed. This stage enables learning of continuous t_{go} estimation throughout the descent.

We now describe the full pipeline to construct the supervised learning dataset. Let $S(t) \in \mathcal{R}^7$ denote the state vector at time t as:

$$\mathcal{S}(t) := [x(t), y(t), z(t), \dot{x}(t), \dot{y}(t), \dot{z}(t), m(t)]^T$$

We adopt the following notation throughout this text. Superscript (i) denotes the trajectory rollout index. For example, $S^{(i)}(t)$ refers to the spacecraft state at time t along the ith trajectory rollout. Subscript j denotes the element index

within a set. For example, $S_j \in \mathcal{X}$ denotes the jth state sample in a dataset $\mathcal{X}$. Let $t_{go} \in \mathcal{R}_{>0}$ denote the time remaining to reach the target, and let m_f represent the final mass of the vehicle at guidance phase termination. Given a state vector S, quadratic polynomial guidance law can be rolled out for a specified range of t_{go} to determine whether the spacecraft reaches the desired terminal conditions under prescribed state and action constraints (Refer Eq.2-6) and the total fuel consumption in the trajectory rollout. This enables the construction of the controllable dataset, $\mathcal{F}(S)$, as:

$$\mathcal{F}(S) = \{(t_{go,k}, m_{f,k}) \quad | \quad k = 1, 2, ..., K\} \tag{18}$$

where K is the cardinality of the set $\mathcal{F}$. For a given state vector S, if the controllable set $\mathcal{F}$ is empty, it indicates that no value of t_{go} leads to convergence of the polynomial guidance to the desired terminal states. It should be noted that the union of the controllable sets corresponding to all admissible states defines the overall controllable region of the proposed guidance algorithm. In this paper, this controllable region is also referred to interchangeably as the region of convergence of the guidance scheme. This usage should not be confused with the term "region of convergence" as it appears in the context of numerical algorithms. Algorithm 1 details the procedure of generating this set. The optimal $t^*_{go}(S)$ for the state vector S representing the time-to-go value that maximizes the final mass (i.e., minimizes fuel consumption) is then computed trivially as:

$$t^*_{go}(S) = \arg \max_{(t_{go}, m_f) \in \mathcal{F}(S)} m_f \tag{19}$$

Algorithm 1 Algorithm for controllable set computation

Input: Dataset S, t_{go}^{min}, t_{go}^{max}, hyperparameters ϵ_1, ϵ_2, ϵ_T, δt_{go}
Output: controllable Dataset $\mathcal{F}(S)$
Define convergence indicator function, $\mathcal{S}_c(S, t_{go})$,for guidance run:

$$\mathcal{I}_c = \begin{cases} 1, & \text{if} \quad \|\boldsymbol{r}(t_f) - \boldsymbol{r}_f\| < \epsilon_1, \quad \|\boldsymbol{v}(t_f) - \boldsymbol{v}_f\| < \epsilon_2, \quad r_3(t) \geq r_f(3), |\theta(t)| \leq \theta_{\lim}, \quad T \in [T_{\min}, T_{\max}], \dot{T} \leq \Delta_T \\ 0, & \text{otherwise} \end{cases}$$

Initialize: $\mathcal{F}(S) \leftarrow \emptyset$, $g_{conv} \leftarrow 0$, $t_{go,i}^{min} \leftarrow t_{go}^{min}$, $t_{go,i}^{max} \leftarrow t_{go}^{max}$
while $(t_{go,i}^{max} - t_{go,i}^{min}) > \epsilon_T$ **do** ▷ Bisection search for computation of minimal t_{go}
    $t_{go}^{current} \leftarrow \frac{t_{go,i}^{min} + t_{go,i}^{max}}{2}$
    Run Guidance simulation with $(S, t_{go}^{current})$ and evaluate $\mathcal{I}_c$
    if $\mathcal{I}_c == 1$ **then**
        $g_{conv} \leftarrow 1$, $t_{go,i}^{max} \leftarrow t_{go}^{current}$
    else
        $t_{go,i}^{min} \leftarrow t_{go}^{current}$
    end if
end while
Record $(t_{go,i}^{min}, m_f)$ in $\mathcal{F}(S)$
Initialize: $t_{go,i}^{max} \leftarrow t_{go}^{max}$
while $(t_{go,i}^{max} - t_{go,i}^{min}) > \epsilon_T$ **do** ▷ Construction of set $\mathcal{F}(S)$)
    Run Guidance Simulation with $(S, t_{go,i}^{max})$ and evaluate $\mathcal{I}_c$
    if $\mathcal{I}_c == 1$ **then**
        Record $(t_{go,i}^{max}, m_{f,i})$ in $\mathcal{F}(S)$)
    else
        continue
    end if
    Update $t_{go,i}^{max} \leftarrow t_{go,i}^{max} - \delta t_{go}$
end while
return $\mathcal{F}(S)$

Stage-1: Initial Perturbation Dataset Stage

Let $\bar{S} \in \mathcal{R}^7$ be the nominal state vector at the start of the terminal descent phase, and $\Sigma \in \mathcal{R}^{7\times7}$ be the covariance matrix representing 3σ dispersions the states. The perturbation in initial mass is assumed to be zero here in Mars landing

scenario, but a nonzero variation can be included if needed. For example, in lunar landing scenarios, the main braking phase also involves propulsion, and its performance can influence the initial mass at the start of the terminal descent phase. Such variations can be captured as part of the 3σ dispersion. The initial perturbation dataset,$\mathcal{X}_{init}$ is generated as

$$\mathcal{X}_{init} := \{\bar{S} + \delta_i, \delta_i \sim \mathcal{N}(0, \Sigma) \quad | \quad i = 1, 2, ..., n\} \tag{20}$$

$\mathcal{X}_{init}$ represents the collection of 3σ perturbed state vectors where each perturbed states are drawn from zero mean gaussian distribution with specified covariance matrix Σ. For each dataset $S_i \in \mathcal{X}_{init}$, we construct the controllable dataset $\mathcal{F}(S_i)$ using algorithm 1 and optimal $t^*_{go}(S_i)$ using Eq.(19). This results in the initial dataset $\mathcal{D}_0$:

$$\mathcal{D}_0 := \{(S_j, t^*_{go}(S_j)) \quad | \quad S_j \in \mathcal{X}_{init}, t^*_{go} \in \mathcal{R}_{>0}, j = 1, 2, ..., |\mathcal{D}_0|\} \tag{21}$$

We ignore any initial state in $\mathcal{D}_0$ for which there is no solution for the polynomial guidance. The collection of such points represents the performance limits of the polynomial guidance and can be used for alternate mission planning.

Stage-2: Trajectory Rollout Dataset Stage

For each dataset $(S_j, t^*_{go}(S_j)) \in \mathcal{D}_0$, a guidance simulation is run producing a trajectory rollout and the resulting trajectory is sampled at fixed intervals to obtain intermediate states for training:

$$\{S^{(j)}(t_k)\}_{k=1}^{K} = \{S^{(j)}(t_1), S^{(j)}(t_2), S^{(j)}(t_3), \ldots, S^{(j)}(t_K)\} \tag{22}$$

where t_k denotes discrete time points sampled at a fixed interval (e.g., every 5 second) along the trajectory. For each sampled states $S^{(j)}(t_k)$, we repeat the process of construction of the controllable dataset $\mathcal{F}(S^{(j)}(t_k))$ using algorithm 1 and optimal $t^*_{go}(S_i)$ using Eq.(19). This results in the timeseries dataset $\mathcal{D}_T$:

$$\mathcal{D}_T := \{(S^{(j)}(t_k), t^{*(j)}_{go}(t_k)) \quad | \quad j = 1, 2, ..., |\mathcal{D}_0|, k = 1, 2, ..., K_j\} \tag{23}$$

where K_j is the number of sampled states in j^{th} trajectory rollout. The full dataset for training is given as:

$$\mathcal{D} := \mathcal{D}_0 \cup \mathcal{D}_T \tag{24}$$

This dataset captures a broad distribution of state vectors, both at the guidance start and during intermediate phases of flight. Using the training dataset, the mapping π as shown in Eq.(17) is learned using a supervised learning approach, where π is modeled by a feedforward neural network. Fig 2 shows the schematics of the neural network architecture. The input to the network is a 6-dimensional state vector:

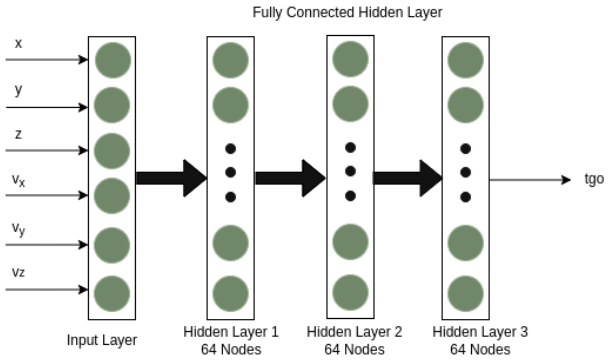


Fig. 2 Schematics of Surrogate Neural Network Model

$$\mathbf{x} = [\mathbf{r}^\top, \mathbf{v}^\top]^\top \in \mathbb{R}^6$$

representing the current position $\mathbf{r} \in \mathbb{R}^3$ and velocity $\mathbf{v} \in \mathbb{R}^3$. The network outputs a scalar value $\hat{t}_{go} \in \mathbb{R}_{\geq 0}$, which estimates the remaining t_{go}. The feedforward neural network consists of three hidden layers, each with 64 neurons and tanh activations. The forward computation is defined as:

$$\mathbf{h}^{(\ell+1)} = \phi^{(\ell)}\left(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell)} + \mathbf{b}^{(\ell)}\right), \quad \ell = 0, 1, 2$$

where: $\mathbf{h}^{(0)} = \mathbf{x} \in \mathbb{R}^6$; $\mathbf{W}^{(\ell)} \in \mathbb{R}^{64 \times d_\ell}$, $\mathbf{b}^{(\ell)} \in \mathbb{R}^{64}$; $\phi^{(\ell)}(\cdot) = \tanh(\cdot)$ for $\ell = 0, 1, 2$ The final output is given by:

$$\hat{t}_{go} = \text{ReLU}(\mathbf{W}^\top \mathbf{h}^{(3)} + b^{(3)})$$

The parameters of the network $\theta = \{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}$ are trained to minimize the regularized mean squared error loss:

$$\mathcal{L}(\pi) = \frac{1}{N} \sum_{i=1}^{N} \left(\pi(\mathbf{x}_i) - t^*_{go,i} \right)^2 + \lambda \sum_{\ell} \left\| \mathbf{W}^{(\ell)} \right\|_F^2$$

where λ is the weight decay coefficient for $\mathcal{L}_2$ regularization, and $\| \cdot \|_F$ denotes the Frobenius norm. The errors are back-propagated to update the weights and bias of the network using the Adam optimizer with mini-batches. The learning rate, batch size, and regularization strength are selected through hyperparameter tuning using validation loss.

V. Simulation Results

The parameters of the problem considered for simulation are

$$\begin{aligned} \mathbf{g} &= \begin{bmatrix} 0 & 0 & -3.71 \end{bmatrix}^T \text{ m/s}^2, \quad m_{\text{wet}} = 1900 \text{ kg}, \\ I_{\text{sp}} &= 225 \text{ s}, \quad T_{min} = 4971.8\text{N}, \quad T_{max} = 13258\text{N} \end{aligned} \tag{25}$$

Comparison with Fuel Optimal Powered Descent Guidance

To evaluate the performance of the proposed QO-LPDG algorithm, two distinct powered descent scenarios are considered: one in which the spacecraft is behind the apriori designated landing site, and another in which it is ahead of the designated site. The simulation results presented in Fig 3 show the time histories of states comparing the performance of the proposed guidance algorithm with Fuel Optimal Powered Descent Guidance (FOPDG) for the first case where the spacecraft is behind the landing site. The lossless convexification-based FOPDG, as discussed in [14], is implemented for comparison. Similarly, the simulation results presented in Fig 4 show the time histories of states for the second case where the spacecraft is ahead of the landing site.

As expected, the proposed QO-LPDG performs less optimally compared to the globally fuel-optimal solution, since the offline optimization identifies a fuel-optimal trajectory only within the class of functions parameterized by quadratic polynomial. It is well known that the true fuel-optimal solution exhibits a bang-bang structure in the acceleration profile, which cannot be captured by quadratic functions. Nevertheless, the proposed guidance successfully identifies the fuel optimal trajectory within this function class, while adhering to the state and control constraints discussed earlier.

Performance comparison with existing Analytical Guidance

As discussed earlier, existing analytical guidance approaches typically compute the t_{go} once at the start of the trajectory and then follow a linear decrement profile. While such strategies are effective in handling initial state dispersion—since the computed t_{go} accounts for initial uncertainties—they lack adaptivity to mid-trajectory perturbations. This limitation becomes critical when unanticipated uncertainties arise during flight due to actuation or navigation errors. For instance, a partial loss of thrust can cause significant deviation from the nominal trajectory. To highlight the benefit of recomputing t_{go} throughout the trajectory, we present two representative simulations. In both cases, the system starts from a nominal state, and errors are artificially injected at 20% and 40% of the trajectory duration to simulate partial failures or navigation/actuation disturbances.

Fig.5 shows the comparison between trajectory evolution where error of 1 km in position and 5 m/s in velocity is artificially injected midway at 20% trajectory duration. While our proposed algorithm is able to adapt to the injected error, one time computation based algorithm (either classical or learning based) fails to converge to desired terminal states. Similarly, Fig.6 shows the comparison between trajectory evolution when error of 800 m in position and 2 m/s in velocity is artificially injected at 40% duration. The adaptive nature of recomputation makes the algorithm robust to uncertainties.

A Monte Carlo analysis is conducted to evaluate the performance improvement achieved by the proposed guidance scheme that continuously recomputes the time-to-go (t_{go}), compared to conventional methods that estimate t_{go} only once at the start. Performance is assessed in terms of the Region of Convergence (ROC)/Controllable region, defined

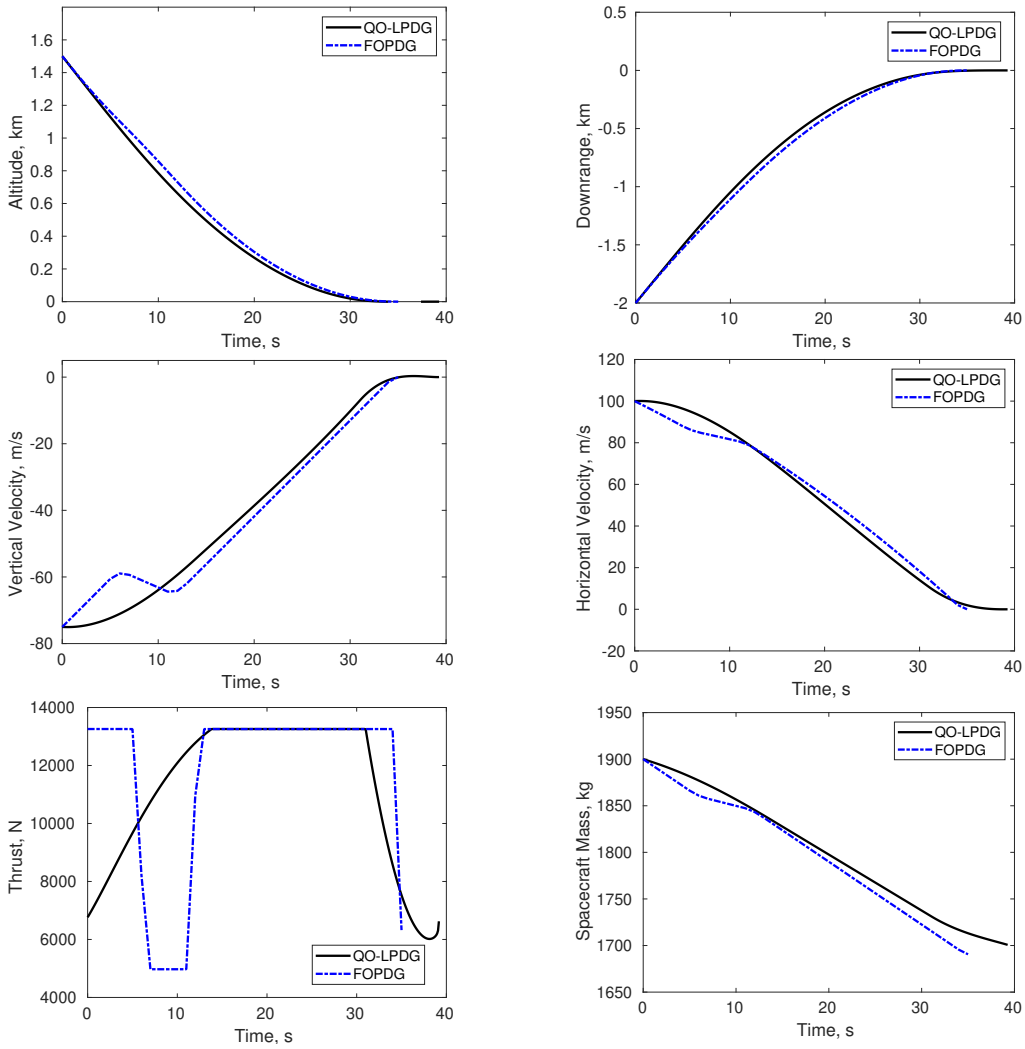


Fig. 3 Comparison of trajectory trace for QO-LPDG and FOPDG; $\mathbf{r_0} = [0\ ,\ -2,\ 1.5]^T$ km, $\mathbf{v_0} = [0\ ,\ 100,\ -75]^T$ m/s, $t_f^{fopdg} = 35s, t_f^{qo-lpdg} = 39s, \Delta_m^{fopdg} = 200kg, \Delta_m^{qo-lpdg} = 210kg$

Table 1 Guidance ROC Comparison With and Without t_{go} Recomputation under Mid-trajectory Disturbances

Disturbance @		W/o Recomputation		With Recomputation	
		Altitude (m)	Downrange (m)	Altitude (m)	Downrange (m)
20%	Min	650	-2000	650	-9000
	Max	1450	-600	9000	-275
40%	Min	300	-1200	300	-7000
	Max	850	-200	6700	-325

as the set of initial perturbations for which the guidance algorithm successfully drives the spacecraft to the desired terminal conditions. Two simulation scenarios are considered, where perturbations in position and velocity are injected at 20% and 40% of the total flight duration, respectively. Fig 7 illustrates the ROC for both guidance schemes at these injection points. The left plot shows the ROC when t_{go} is not recomputed after the perturbation, while the right plot shows the ROC when the proposed continuous recomputation strategy is used. As the spacecraft progresses towards the target, the ROC naturally shifts rightward from 20% and 40% flight time. It is evident that without recomputation, the

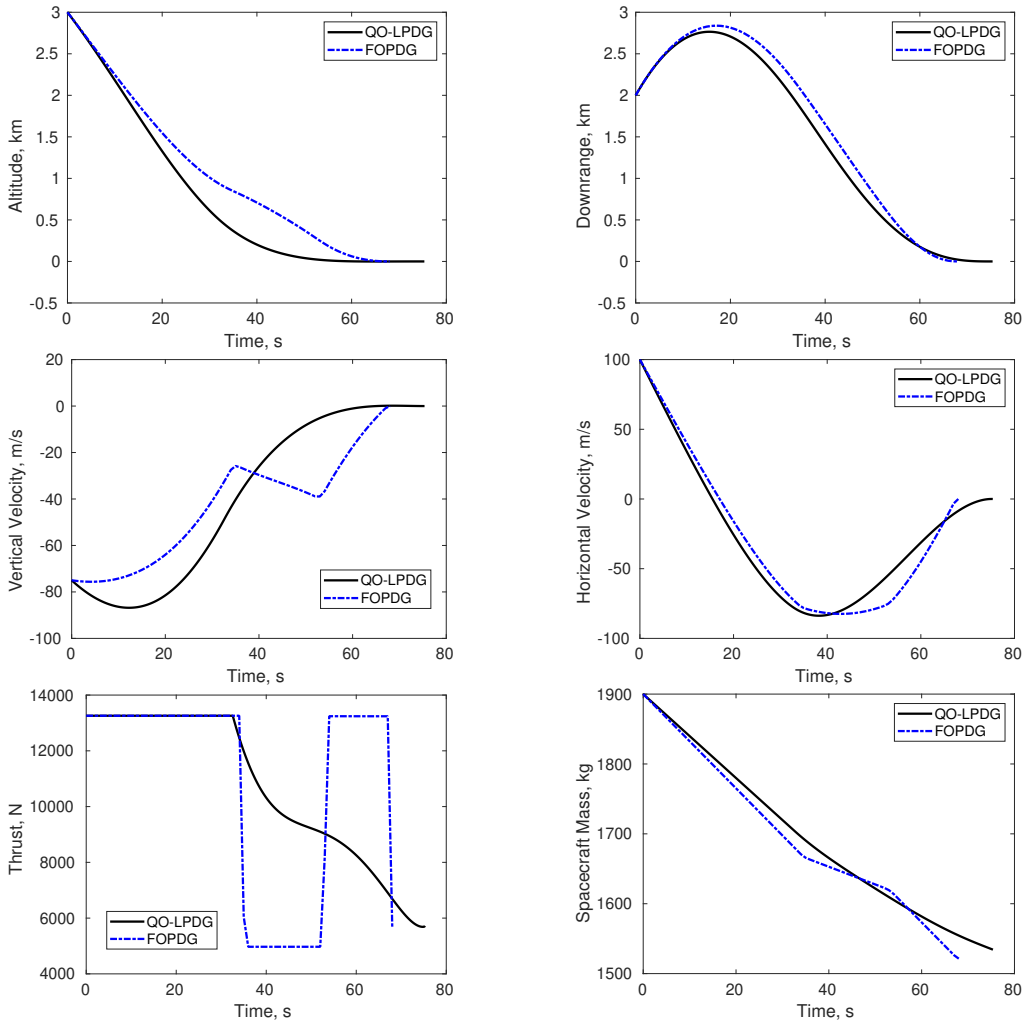


Fig. 4 Comparison of trajectory trace for QO-LPDG and FOPDG; $\mathbf{r_0} = [0\ ,\ 2,\ 3]^T$ km, $\mathbf{v_0} = [0\ ,\ 100,\ -75]^T$ m/s, $t_f^{fopdg} = 68s, t_f^{qo-lpdg} = 75s, \Delta_m^{fopdg} = 365kg, \Delta_m^{qo-lpdg} = 380kg$

ROC shrinks significantly—indicating reduced robustness, as the guidance fails to adapt to mid-trajectory disturbances. In contrast, with continuous recomputation, the ROC remains nearly unchanged, clearly demonstrating the enhanced robustness of the proposed approach in adapting to dynamic uncertainties during descent. Fig 8 shows the relative comparison of ROC for both the simulations. The guidance scheme with time-to-go recomputation exhibits significantly greater adaptivity to mid-trajectory perturbations compared to the scheme without recomputation owing to much larger ROC. Table 1 shows the ROC statistics for both the cases.

Finally, we carry Monte-carlo analysis for the 3D case for the proposed algorithm. Fig9 shows the trajectory trace for various perturbed initial conditions in both position and velocity. In particular, dispersion of ±1 km is considered in position and ±10 m/s is considered in velocity. The trained system clearly manages to drive the spacecraft to desired states under realistic state and action constraints.

VI. Conclusion

This paper proposed a novel Quasi-Optimal Learning based Powered Descent Guidance framework that combines the analytical structure of optimal feedback guidance with data-driven learning to ensure quasi-optimal fuel performance under state and action constraints. Unlike existing methods that estimate the time of flight only at the start of the guidance, a surrogate learning-based model is developed to estimate it continuously along the trajectory. This model is

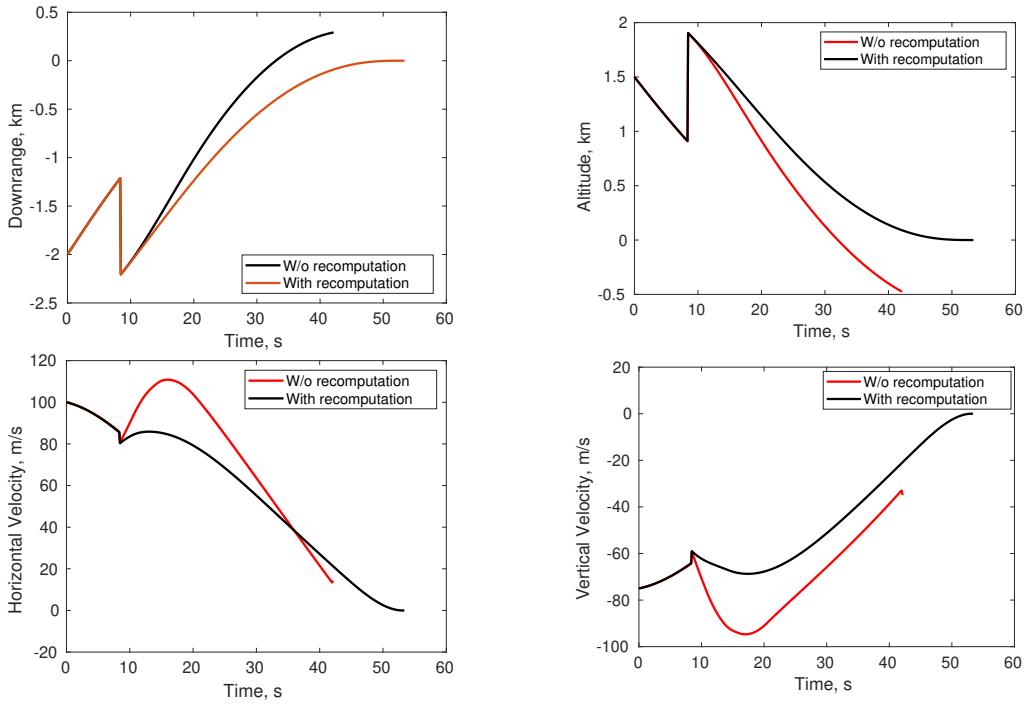


Fig. 5 Comparison of trajectory trace for QO-LPDG and existing analytical guidance; $\mathbf{r_0} = [0\ ,\ -2,\ 1.5]^T$ km, $\mathbf{v_0} = [0\ ,\ 100,\ -75]^T$ m/s, $\Delta_y = -1km, \Delta_z = 1km, \Delta_{vy} = -5m/s, \Delta_{vz} = 5m/s$

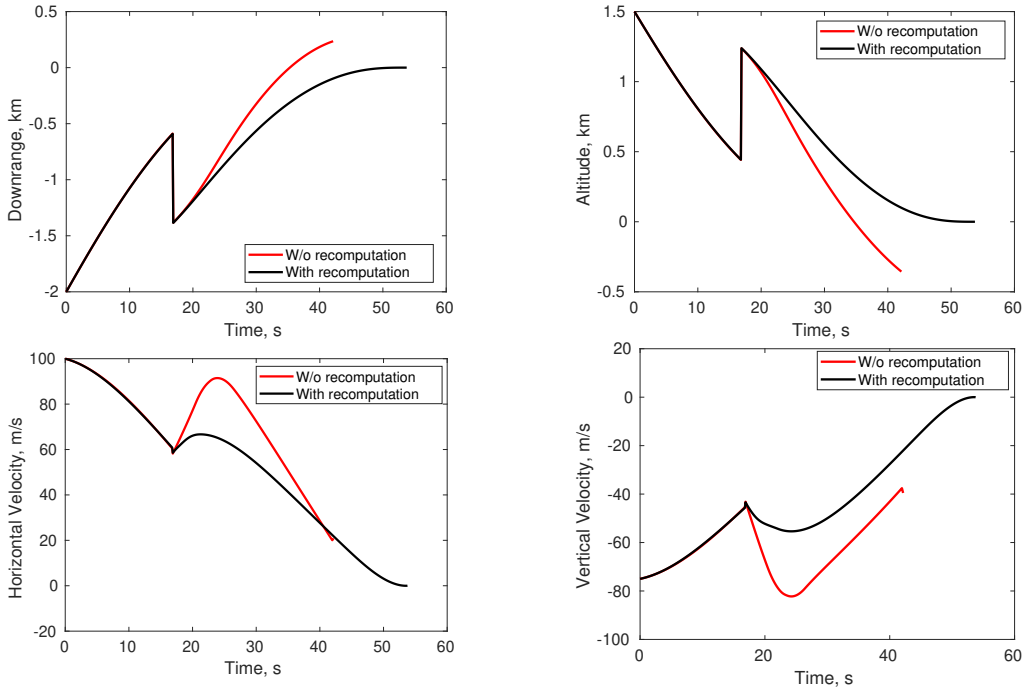


Fig. 6 Comparison of trajectory trace for QO-LPDG and Comparison of trajectory trace for QO-LPDG and existing analytical guidance; $\mathbf{r_0} = [0\ ,\ -2,\ 1.5]^T$ km, $\mathbf{v_0} = [0\ ,\ 100,\ -75]^T$ m/s, $\Delta_y = -0.8km, \Delta_z = 0.8km, \Delta_{vy} = -2m/s, \Delta_{vz} = 2m/s$

trained using simulation data to ensure that the generated trajectories satisfy system constraints and selects a fuel-optimal

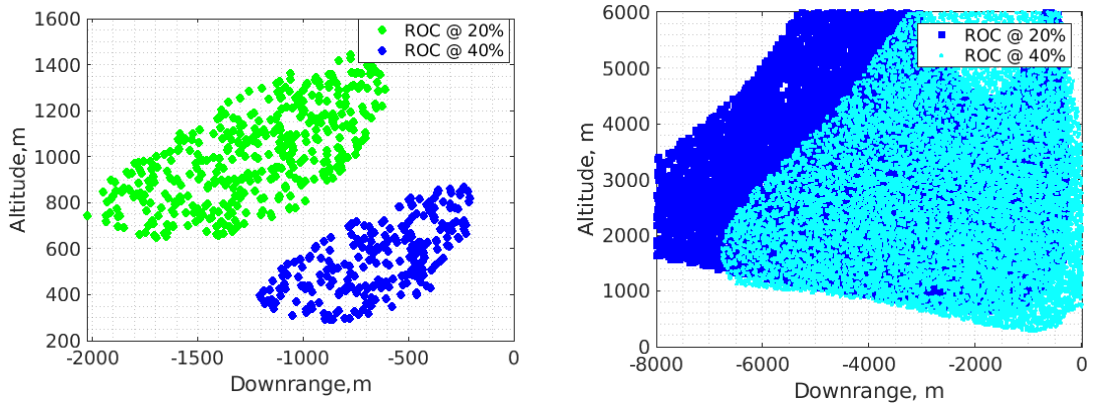


Fig. 7 Guidance ROC: a) left plot shows the ROC at 20% and 40% flight without recomputation b) right plot shows the ROC at 20% and 40% flight with proposed recomputation

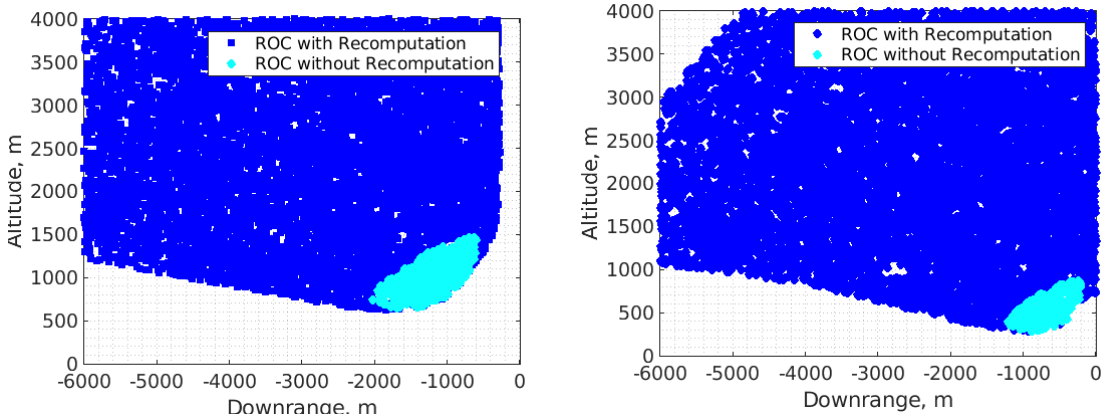


Fig. 8 Guidance ROC Comparision: a) left plot shows the comparison of ROC at 20% of flight. b) right plot shows the comparison of ROC at 40% of flight.

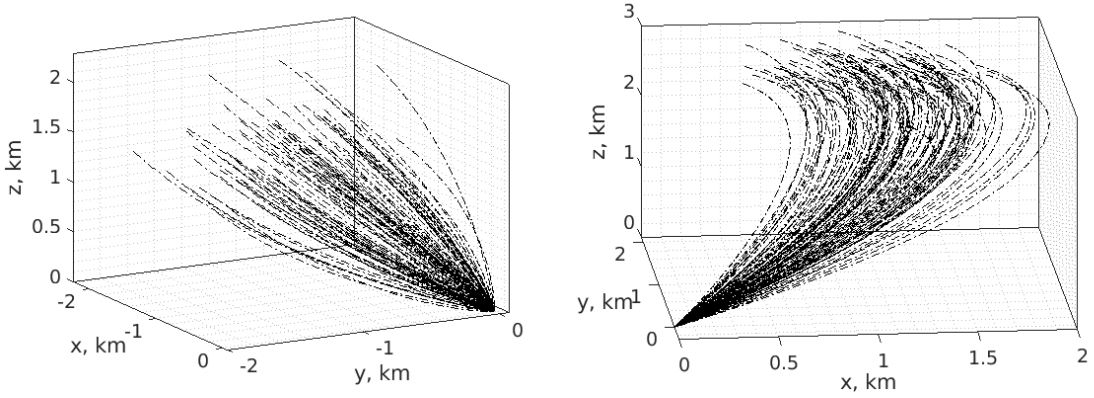


Fig. 9 3D Monte-carlo Simulation Trajectory Trace: a) left plot: spacecraft approaching the landing site from behind b) spacecraft approaching landing site from ahead

solution for each initial condition from a set of trajectories defined by the chosen function class. In the future, our goal is to relax the quadratic polynomial assumption in the acceleration parameterization and utilize surrogate model approach to directly learn true fuel optimal solution that is semi-analytic in nature.

Acknowledgments

We convey our sincere gratitude to the U R Rao Satellite Center (URSC) for encouraging and supporting this research. We wish to gratefully acknowledge the excellent review committee at URSC for reviewing this work and providing valuable feedback. This research work was carried out as a part of GNC development for the Mars Landing Mission.

References

[1] Rallapalli, A., Kumar, S., Kumar, G. B., Majumder, C. G., Chakrabarti, D., and Rijesh, M., "Landing Profile and Powered Descent Strategy for Chandrayaan-3," *Journal of Aerospace Sciences and Technologies*, 2024, pp. 197–207.

[2] Ueda, S., Ito, T., Yokota, K., Sakai, S., Ishida, T., Miyazawa, Y., Kushiki, K., Fukuda, S., and Sawai, S., "On-orbit operation results of the powered descent guidance algorithm for pinpoint lunar landing," *Acta Astronautica*, Vol. 236, 2025, pp. 47–61. https://doi.org/10.1016/j.actaastro.2025.06.039.

[3] Farley, K. A., Williford, K. H., Stack, K. M., Bhartia, R., Chen, A., de la Torre, M., Hand, K., Goreva, Y., Herd, C. D., Hueso, R., et al., "Mars 2020 mission overview," *Space Science Reviews*, Vol. 216, No. 8, 2020, p. 142. https://doi.org/10.1007/s11214-020-00762-y.

[4] Klumpp, A. R., "Apollo lunar descent guidance," *Automatica*, Vol. 10, No. 2, 1974, pp. 133–146. https://doi.org/10.1016/0005-1098(74)90019-3.

[5] Sostaric, R., and Rea, J., "Powered descent guidance methods for the moon and mars," *AIAA guidance, navigation, and control conference and exhibit*, 2005, p. 6287. https://doi.org/10.2514/6.2005-6287.

[6] Furfaro, R., Scorsoglio, A., Linares, R., and Massari, M., "Adaptive generalized ZEM-ZEV feedback guidance for planetary landing via a deep reinforcement learning approach," *Acta Astronautica*, Vol. 171, 2020, pp. 156–171. https://doi.org/10.1016/j.actaastro.2020.02.051.

[7] Ploen, S., Acikmese, B., and Wolf, A., "A comparison of powered descent guidance laws for mars pinpoint landing," *AIAA/AAS astrodynamics specialist conference and exhibit*, 2006, p. 6676. https://doi.org/10.2514/6.2006-6676.

[8] Lu, P., "Augmented apollo powered descent guidance," *Journal of Guidance, Control, and Dynamics*, Vol. 42, No. 3, 2019, pp. 447–457. https://doi.org/10.2514/1.G004048.

[9] Guo, Y., Hawkins, M., and Wie, B., "Optimal feedback guidance algorithms for planetary landing and asteroid intercept," *AAS/AIAA astrodynamics specialist conference*, AAS, 2011, pp. 2011–588.

[10] Guo, Y., Hawkins, M., and Wie, B., "Waypoint-optimized zero-effort-miss/zero-effort-velocity feedback guidance for Mars landing," *Journal of Guidance, Control, and Dynamics*, Vol. 36, No. 3, 2013, pp. 799–809. https://doi.org/10.2514/1.58098.

[11] Zhou, L., and Xia, Y., "Improved ZEM/ZEV feedback guidance for Mars powered descent phase," *Advances in Space Research*, Vol. 54, No. 11, 2014, pp. 2446–2455. https://doi.org/10.1016/j.asr.2014.08.011.

[12] Lu, P., "Propellant-optimal powered descent guidance," *Journal of Guidance, Control, and Dynamics*, Vol. 41, No. 4, 2018, pp. 813–826. https://doi.org/10.2514/1.G003243.

[13] Lu, P., and Callan, R., "Propellant-optimal powered descent guidance revisited," *Journal of Guidance, Control, and Dynamics*, Vol. 46, No. 2, 2023, pp. 215–230. https://doi.org/10.2514/1.G007214.

[14] Acikmese, B., and Ploen, S. R., "Convex programming approach to powered descent guidance for mars landing," *Journal of Guidance, Control, and Dynamics*, Vol. 30, No. 5, 2007, pp. 1353–1366. https://doi.org/10.2514/1.27553.

[15] Açıkmeşe, B., Carson, J. M., and Blackmore, L., "Lossless convexification of nonconvex control bound and pointing constraints of the soft landing optimal control problem," *IEEE transactions on control systems technology*, Vol. 21, No. 6, 2013, pp. 2104–2113. https://doi.org/10.1109/TCST.2012.2237346.

[16] Dueri, D., Açıkmeşe, B., Scharf, D. P., and Harris, M. W., "Customized real-time interior-point methods for onboard powered-descent guidance," *Journal of Guidance, Control, and Dynamics*, Vol. 40, No. 2, 2017, pp. 197–212. https://doi.org/10.2514/1.G001480.

[17] Sagliano, M., "Generalized hp pseudospectral-convex programming for powered descent and landing," *Journal of Guidance, Control, and Dynamics*, Vol. 42, No. 7, 2019, pp. 1562–1570. https://doi.org/10.2514/1.G003731.

[18] Kumar, S., Rallapalli, A., Kakula, A. K., and GVP, B. K., "Powered Descent Trajectory Optimization of Chandrayaan-3 using Radau Collocation and Controllable Sets," *arXiv preprint arXiv:2511.03594*, 2025. https://doi.org/10.48550/arXiv.2511.03594.

[19] Briden, J., Gurga, T., Johnson, B. J., Cauligi, A., and Linares, R., "Improving computational efficiency for powered descent guidance via transformer-based tight constraint prediction," *AIAA SCITECH 2024 Forum*, 2024, p. 1760. https://doi.org/10.2514/6.2024-1760.

[20] You, S., Wan, C., Dai, R., and Rea, J. R., "Learning-based onboard guidance for fuel-optimal powered descent," *Journal of Guidance, Control, and Dynamics*, Vol. 44, No. 3, 2021, pp. 601–613. https://doi.org/10.2514/1.G004928.

[21] Gaudet, B., Linares, R., and Furfaro, R., "Deep reinforcement learning for six degree-of-freedom planetary landing," *Advances in Space Research*, Vol. 65, No. 7, 2020, pp. 1723–1741. https://doi.org/10.1016/j.asr.2019.12.030.

[22] D'Ambrosio, A., and Furfaro, R., "Learning fuel-optimal trajectories for space applications via Pontryagin neural networks," *Aerospace*, Vol. 11, No. 3, 2024, p. 228. https://doi.org/10.3390/aerospace11030228.

[23] Chakrabarti, D., Kumar, S., Rallapalli, A., Rijesh, M., and Kumar, G. B., "Convex Decision Boundary Design for Guidance Feasibility Check During Powered Descent Phase of Chandrayaan-3 Lander," *IFAC-PapersOnLine*, Vol. 57, 2024, pp. 119–124. https://doi.org/10.1016/j.ifacol.2024.05.021.

[24] Srinivasa, T. R., and Kumar, S., "Solving Infinite-Horizon Optimal Control Problems using the Extreme Theory of Functional Connections," *arXiv preprint arXiv:2510.27187*, 2025. https://doi.org/10.48550/arXiv.2510.27187.